

Agentic model routing: tau-bench study

(cascade vs predictive routing)

Internal technical report. Companion to `docs/paper/liteforge-routing.tex` ("When Does Model Routing Help?") and `docs/provider-cost-quality-study.md`. All numbers below are N=50 on the full tau-bench airline test split, agent temperature 0, 3 trials/task, with a param-scaled compute-cost proxy for the (free-via-gateway) OSS tiers. Code: `scripts/eval/agentic/`; results: `scripts/eval/data/agentic_*.json`.

TL;DR

1. **Predictive routing does not transfer to agentic tasks.** Both shipped routers (the `router-bert` difficulty classifier and the MF/RouteLLM-port over `bge-m3`) score at or below random APGR on tau-bench (MF significantly negative). Agentic difficulty is not legible from the opening user turn: "change my flight" reads easy, the episode is hard.
2. **Escalate-on-failure (cascade) is the right frame.** An oracle cascade (run cheap, escalate only when the cheaper tier actually fails) reaches 0.727 quality (nemotron pool) and 0.753 (gemma pool), both *above* always-using-the-strongest model (0.628), because it keeps the cheap tier's wins on tasks where the strong model itself fails.
3. **The best cheap, realistic trigger is self-consistency, not an LLM judge.** Running the cheap tier a few times and escalating on disagreement recovers 84% (nemotron) to 98% (gemma) of all-strong quality at roughly half the cost, with no verifier.
4. **LLM-judge verification is calibration-bound and rarely worth it.** Only a strong, calibrated judge (glm-5.2, recall ~0.6) helps; small judges fail (too lenient or too harsh); none reach the oracle ceiling because silent failures (a wrong booking that *looks* right) cannot be verified from the transcript. The good judge is also the escalation target, so its cost win is small.
5. **The biggest lever is a strong cheap base model.** Swapping the weak tier from `nemotron-3-nano:30b` to `gemma4:31b`, the "weak" tier (0.581) beats the 397B "medium" tier (0.546) at 11x lower cost, and gemma alone delivers 93% of all-strong quality at 6% of the cost, largely removing the need to route at all.

What we built

A real, sandboxed agentic harness (not multiple-choice), faithful to the official tau-bench protocol, with four tools:

- `scripts/eval/agentic/run_agent.py` - OpenAI-compatible tool-calling loop against the LiteLLM gateway: `POOLS` (tier definitions), `call_chat` (with

finish_reason/cost capture and a price fallback), OLLAMA_PRICES (the compute-cost proxy), and the PROTO cache-version tag.

- `scripts/eval/agent/benches/taubench.py` - tau-bench adapter: real Python tools + an LLM-simulated user, graded by the env's DB-state reward. Matches the official ToolCallingAgent (one action = first tool_call per turn, max_num_steps=30), a Sonnet user-sim, a generic agent scaffold applied to every tier, full transcript capture, and --start for sharding.
- `scripts/eval/agent/routing_eval.py` - builds the task x tier success/cost matrix and scores predictive routers (router_runner.BertRunner, MfRunner) with binary APGR (stats.fast_apgr), 3-tier decision points, oracle/random/all-tier baselines, and a power calc.
- `scripts/eval/agent/cascade_eval.py` - simulates cascade strategies from the cache plus a parallel LLM-judge verifier (--judge-model); reports the cost-quality table and judge precision/recall.
- Helpers: `show_transcript.py` (read an episode), `rebalance.py` (drain shard tails).

The diagnosis journey (what we did)

The first runs showed a **tier inversion**: the stronger model scored worse than the weaker one. Capturing transcripts and reading them traced this to three issues, none of them capability:

1. **max_tokens truncation of reasoning models (the dominant bug).** At `max_tokens=4096` a verbose/thinking model (opus, in the early gemini pool) spent its budget on reasoning and the response was cut off before it emitted a tool call, producing empty turns that looped to the step cap. A probe confirmed `truncated_calls` dropped 5 -> 0 when raised to 16384.
2. **A too-weak user simulator.** Using haiku as the tau user-sim, it broke character (it once replied "reservation confirmed!" right after a tool error and ended the episode) and approved constraint-violating bookings. We switched the user-sim to Sonnet.
3. **Harness non-parity.** We initially executed all parallel tool calls per turn and capped steps at 20; the official agent takes only the first tool call per turn with a cap of 30. We matched it exactly.

These fixes are versioned through the episode cache tag (PROTO, now v5). With the corrected harness, the OSS ollama pool ranks monotonically and the inversion is gone; what remains are the genuine, more interesting findings below.

Results: cost-quality by pool (N=50 airline)

Quality = mean tau reward (DB-state correctness); cost = param-scaled \$/task proxy; %strongQ/C are relative to always-using-the-strong-tier.

strategy

97b
)

all-
stro
ng
(gl
m-
5.2
)

quality
(95%
CII)

.43,
0.67]

0.628
[0.51,
0.74

\$/
task

0.083

%
strong
Q

10%

%
strong
C

10%

strategy

quality
(95% CI)

\$ / task

%
storing Q

%
storing C

random-tier

0.561

0.057

89%

69%

cas
cad
e:
ora
cle
(cei
lin
g)

0.727

0.088

116%

106%

cas
cad

0.831

0.077

78

67

strategy

quality (95% CI)

\$ / task

% strong Q

% strong C

give-up signal

487

056

%

%

cascade: self-consistency

0.526

0.411

84%

49%

cascade: judge (glm-5.2:cloud)

0.600

0.608

96%

81%

pre

0

0

9

8

strategy	quality	\$ / task	% strong	% success
	(95% CI)		Q	C
dict				
:	.	.		
router-bert	573	072	1%	6%
predict				
:	0	0		
mf	.	.	8	5
(RouteLLM-port)	500	049	0%	9%

Routable (strong succeeds where weak fails) = $9/50 = 0.18$. Binary APGR: oracle 1.00, random ~0, router-bert -0.14, **mf -0.44** (both at or below random).

Pool B: gemma-weak (a strong cheap base; weak=0.581)

st r a t e g y	q			
	u			
	a			
	l			
	i		%	%
	t	\$	s	s
	y	/	t	t
	(	t	r	r
	9	a	o	o
	5	s	n	n
	%	k	g	g
	C		Q	C
	I			
	)			
al	0			
l-	.			
w	5			
e	8			
a	1			
k				
(	[			
g	0	0		
e	.	.	9	6
m	4	0	3	%
m	7	0	%	
a	,	5		
4				
:	0			
3	.			
1	7			
b	0			
)	]			
al	0	0	8	6
l-	.	.	7	8
m	5	0	%	%
e	4	5		
d	6	6		
i				
u	[			
m	0			

strategic

quality (95% CI)

\$ / task

% strong Q

% strong C

(qwenn3.5:397b)

.43, 0.67]

all-strong ongoing (glm-5.2

0.628 [0.51, 0

0.083

100%

100%

Statement	Quantity (95% CI)	\$ task	% strong Q	% strong C
) r a n d o m - t i e r	. 7 4]	0 . 5 8	9 4 %	6 6 %
c a s c a d e: o r a c l e (	0 . 7 5 3 [0 . 6 4 ,	0 . 0 7 1	1 2 0 %	8 6 %

strategic	quality (95% CI)	\$ / task	% strong Q	% strong C
ceiling)	0.85]			
cas- caded: give- up sign- al	0.567	0.40	90%	47%
cas-	0.6	0.0	98%	42

strategy

quality (95% CI)

\$ / task

% strong Q

% strong C

cadence: self-
consciousness

13

35

%

casualty: judge

0.614

0.046

98%

55%

strategy

quality (95% C I)

\$ / task

% strong Q

% strong C

(glmm - 5.2 : confidence)

predicted route

0.540

0.073

86%

worse than all -

st
r
a
t
e
g
y

q
u
a
l
i
t
y

(
9
5
%

C
I
)

\$
/
t
a
s
k

%
s
t
r
o
n
g
Q

%
s
t
r
o
n
g
C

w
e
a
k

$\text{Routable} = 9/50 = 0.18$. Note `gemma4:31b weak (0.581)` > `qwen3.5:397b medium (0.546)`: the 397B "medium" tier is Pareto-dominated by a 31B model.

The verifier-strength sweep

We ran the judge cascade with three verifiers to test whether transcript-verification scales with judge size. It is calibration-bound, not size-monotonic:

judge

s
i
z
e

p
r
e
c
i
s
i
o
n

r
e
c
a
l
l

gemm
a3:27
b-
cloud

2
7
B

0
.
5
0
-
0
.
5
8

0
.
0
2
-
0
.
0
7
(
t
o
o

l
e
n
i
e
n
t
)

qwen
3:14b

1
4
B

0
.
6

0
.
4
4

judge

s
i
z
e

p
r
e
c
i
s
i
o
n

r
e
c
a
l
l

4

(
h
a
r
s
h
,

m
a
n
y

f
a
l
s
e

e
s
c
a
l
a
t
i

judge

s
i
z
e

p
r
e
c
i
s
i
o
n

r
e
c
a
l
l

o
n
s
)

0
.
5
5
-
0
.
6
0

(
b
e
s
t
)

glm-
5.2:cl
oud

s
t
r
o
n
g

t
i
e
r

0
.
6
7
-
0
.
6
8

oracle
(perfe
ct
detect
or)

-

1
.
0
0

1
.
0
0

pre
s
i
z
e

Why predictive routing fails here (and where it works)

Recommendation

1. **Default to a strong cheap base model.** `gemma4:31b` is the standout: it beats a 397B tier at a fraction of the cost and reaches 93% of the strongest tier's quality alone. This is the single largest lever and it agrees with the cross-provider study (`provider-cost-quality-study.md`, `gemma4:31b` 0.865 = 96% of

Sonnet).

2. **Escalate on failure, do not predict difficulty up front.** Cascade beats predictive routing, and an oracle cascade beats even always-strong.
3. **Use self-consistency as the cheap escalation trigger.** Run the base a few times, escalate on disagreement: ~half the cost of always-strong at 84-98% of its quality, no verifier needed.
4. **Reserve an LLM-judge verifier for cases that justify a strong, calibrated judge.** A small or mis-calibrated judge adds cost without recall; even a strong judge leaves the silent-failure headroom on the table.
5. **Skip the predictive difficulty router for agentic routing** (keep it for QA-like domains).

Methodology notes and gotchas

- Reasoning/verbose models need a generous `max_tokens` (we use 16384 for the agent, 2048 for the judge) or they truncate before emitting their tool call / verdict; parse the explicit final `VERDICT:` line and fail-open so a truncated judgment never spuriously escalates.
- Gateway model ids matter: the strong tier is `glm-5.2:cloud` (the bare `glm-5.2` 400s and, with fail-open handling, silently degrades a judge to "never escalate").
- Some models are unloaded/cold on the gateway and time out (`qwen3.6:27b`, `granite4.1:30b`, `nemotron3:33b`); responsive sub-40B judges include `gemma3:27b-cloud`, `gpt-oss:20b-cloud`, `qwen3:14b`.
- Per-task tau variance is large (frontier pass¹ is ~0.4-0.55); N=5 pilots can show spurious inversions. The full 50-task split (the whole airline test set) is what these conclusions rest on. retail is a quality ceiling (all tiers ~1.0) and carries no routing signal.
- Cost axis: OSS tiers are free via the gateway, so the economic axis uses a param-scaled \$/Mtok proxy applied to measured tokens (`OLLAMA_PRICES`); only the monotonic ordering by tier matters for the routing shape, and the constants are a documented assumption.